STATISTICS WITH R PROGRAMMING

ASSIGNMENT 1

Design and Develop a Scalable R Data Representation Framework for a Smart Retail Sales and Inventory Analytics System

SUBMISSION DETAILS	INFORMATION
Course	Statistics with R Programming
Course Outcome	CO1 – R data representations and structures
Assignment	Assignment 1 – Complete Worked Solution
Student Name	P.VIGNESH
Register Number	192525013
Department / Class	BTech AIMI
Submitted To	Dr.S.Radhika
Date	9-11-2026

CONTENTS

- Executive Summary and Assumptions
- Task 1 – Business Enquiry and Architecture Blueprint
- Task 2 – Store and Product Core Data
- Task 3 – Coercion-Safe Sales Preparation
- Task 4 – Sales Matrix Engine
- Task 5 – Multi-Dimensional Sales Array
- Task 6 – Lists, Factors and Data Frames
- Task 7 – Object-Oriented R Classes
- Task 8 – Reusable R Function Library
- Task 9 – Integration and Testing
- Task 10 – Scalability Evaluation
- Task 11 – Changed Requirement
- Task 12 – Documentation and Packaging
- Consolidated Reproducible R Script
- Conclusion and References

EXECUTIVE SUMMARY AND ASSUMPTIONS

This report designs an integrated R data-representation framework for a national retail chain. It stores store and product master information, validates incoming sales feeds, calculates revenue, represents sales across products, months and channels, and provides reusable functions and object-oriented behaviour. Vectors, lists, factors, matrices, arrays and data frames are combined because each structure has a different purpose.

The assignment begins with approximately 500 stores and 40 product categories, expects growth to 10,000 stores, and later changes the requirement to 10,000 stores, 100 categories and three sales channels. Synthetic values are used for demonstration. They are not actual retailer records.

Design assumptions

- Store and product IDs are character keys, not quantities; leading zeros and prefixes must be preserved.
- Footfall, stock and units sold are non-negative whole-number counts.
- Prices and revenue are numeric doubles in this prototype.
- Promotion and store-open status are logical TRUE/FALSE values.
- Region, category, store format and channel are controlled categorical variables.
- NA means unknown or missing; zero means an observed quantity of zero.
- The core solution uses base R so it can run without installing extra packages.

Overall architecture

Layer	R representation	Purpose
Identifiers	Character vectors	Stable store_id and product_id keys
Quantities/prices	Integer and double vectors	Footfall, stock, units and price
Flags	Logical vectors	Open and promotion status
Controlled categories	Factors	Region, category, channel and risk
Complex individual record	Nested list	Heterogeneous store/product details
Analysis-ready records	Data frame	Rows of sales or master information
Two-dimensional analysis	Matrix	Store-by-product unit totals and revenue
Multi-dimensional analysis	Array	Store × product × month × channel
Business entity	S3 / S4 class	Validation and reusable business methods

TASK 1 – FORMULATE THE BUSINESS ENQUIRY AND CREATE THE DATA ARCHITECTURE BLUEPRINT

1.1 Business requirements

The platform must integrate store registration, product catalogue, daily footfall, unit prices, stock levels, promotion flags, monthly sales and sales-channel information. It must support data entry, validation, updating, aggregation, reporting and future expansion.

Category	Example fields	Expected operations	Representation
Store	store_id, name, region, format, open	Register, update, filter, group	Character/logical vectors, factor, data frame
Product	product_id, name, category, price	Catalogue lookup, price update	Character/double vectors, factor, data frame
Inventory	stock_units, reorder_level	Update stock, reorder alerts	Integer vectors/data frame
Footfall	daily_footfall, date	Daily total, average, trend	Integer vector/data frame
Sales	units_sold, revenue, month	Revenue and aggregation	Integer/double, matrix, array
Channel	in_store, online, mobile_app	Channel comparison	Factor and array dimension

1.2 Relationships

- One store can have many sales records.
- One product can appear in sales records from many stores.
- A store can sell many products and a product can be sold by many stores; this is a many-to-many relationship represented by a sales fact table.
- Each sales observation is linked to one store, product, month and channel.
- Store and product master tables provide descriptive attributes; sales records refer to them using IDs.

1.3 Architecture workflow

Stage	Input	Representation / operation	Output
Raw ingestion	CSV/API feed	Raw data frame; explicit type conversion	Raw sales
Validation	Raw values	Type, missing, range and category checks	Accepted and rejected rows
Master layer	Store/product data	Vectors, factors, lists and data frames	Validated master tables
Calculation	Units and prices	Matrix multiplication / aggregation	Revenue summaries
Analytics	Store/product/month/channel	Named arrays and apply()	Sales cube
Business logic	Validated entities	S3/S4 classes and functions	Reusable objects
Reporting	Validated data and summaries	Data frames and lists	Management report

1.4 Reproducible R session

```
# Start every run with a clean, reproducible session
rm(list = ls())
set.seed(2026)
options(stringsAsFactors = FALSE)
cat("R version:", R.version.string, "\\n")
cat("Working directory:", getwd(), "\\n")
sessionInfo()
# Base R is sufficient for the core implementation.
```

1.5 Experimental enquiry questions

1. What happens when numeric, integer, character and logical values are combined in one atomic vector?
2. Which representation makes store/product updates safe and understandable?
3. How can negative prices, missing IDs and invalid categories be detected before analysis?
4. Can a named matrix calculate revenue correctly while preserving product identifiers?
5. How can an array summarize sales independently by month and channel?
6. How do execution time and object size change when stores increase from 500 to 10,000?

1.6 Capacity estimation

For a store-by-product matrix, cells = number of stores × number of product categories. A double is commonly 8 bytes, so the raw payload estimate is cells × 8 bytes. This estimate excludes object headers, dimnames, identifiers, data-frame overhead and other metadata.

Scenario	Stores	Categories/dimensions	Cells	Raw double payload
Initial	500	40	20,000	160,000 bytes ≈ 0.153 MiB
Growth	10,000	40	400,000	3,200,000 bytes ≈ 3.05 MiB
Changed	10,000	100	1,000,000	8,000,000 bytes ≈ 7.63 MiB
12 months × 3 channels	10,000	100 × 12 × 3	36,000,000	288,000,000 bytes ≈ 274.66 MiB

The final line estimates only numeric payload for a dense array. Actual memory use is larger. A production system should measure object.size() and consider a database or sparse representation when many combinations have no sales.

1.7 Acceptance criteria

Criterion	Acceptance test
Type safety	IDs character; prices numeric; flags logical; counts non-negative.
Validation	Missing IDs, negative values and unknown categories are detected.
Traceability	Matrix/array dimensions retain store/product/month/channel names.
Correctness	Revenue equals units × price; totals reconcile.
Maintainability	Reusable functions and documented object structures.
Scalability	Same design supports 500 and 10,000 stores.
Reproducibility	Fresh R session rebuilds data and tests using set.seed().

TASK 2 – DESIGN A ROBUST STORE AND PRODUCT CORE-DATA REPRESENTATION

2.1 Store and product objects

IDs are character because they may contain letters or leading zeros. Names are character. Footfall and stock are whole-number counts. Price is double. Open and promotion flags are logical. Region and category are factors when the allowed values are controlled.

```
stores <- data.frame(
  store_id = c("S001","S002","S003","S004","S005"),
  store_name = c("Central","Marina","Airport","City Mall","West End"),
  region = factor(c("North","South","East","South","West"),
                  levels=c("North","South","East","West")),
  store_format = factor(c("High Street","Mall","Airport","Mall","High Street")),
  daily_footfall = as.integer(c(420,510,680,390,275)),
  is_open = c(TRUE,TRUE,TRUE,FALSE,TRUE)
)

products <- data.frame(
  product_id = c("P001","P002","P003","P004"),
  product_name = c("Rice 5kg","Cooking Oil 1L","Tea 250g","Soap Pack"),
  category = factor(c("Grocery","Grocery","Beverage","Personal Care")),
  unit_price = as.double(c(320,145.5,92,78)),
  reorder_level = as.integer(c(30,25,20,40))
)
str(stores)
str(products)
class(stores$store_id); typeof(stores$store_id)
class(products$unit_price); typeof(products$unit_price)
```

2.2 Validation logic

```
validate_stores <- function(x) {
  problems <- character()
  required <- c("store_id","store_name","region",
                "daily_footfall","is_open")
  missing_cols <- setdiff(required,names(x))
  if(length(missing_cols)) problems <- c(problems,"Required columns missing")
  if(anyNA(x$store_id) || any(x$store_id=="",na.rm=TRUE))
    problems <- c(problems,"Store ID missing or blank")
  if(anyDuplicated(x$store_id)) problems <- c(problems,"Duplicate store IDs")
  if(!is.numeric(x$daily_footfall))
    problems <- c(problems,"Footfall must be numeric")
  else if(any(x$daily_footfall<0,na.rm=TRUE))
    problems <- c(problems,"Footfall cannot be negative")
  if(!is.logical(x$is_open)) problems <- c(problems,"is_open must be logical")
  list(valid=length(problems)==0,problems=problems)
}
```

```
validate_products <- function(x) {
  problems <- character()
  if(anyNA(x$product_id) || anyDuplicated(x$product_id))
    problems <- c(problems,"Product ID missing or duplicated")
  if(!is.numeric(x$unit_price) || any(x$unit_price<0,na.rm=TRUE))
    problems <- c(problems,"Price must be numeric and non-negative")
  if(any(x$reorder_level<0,na.rm=TRUE))
    problems <- c(problems,"Reorder level cannot be negative")
  list(valid=length(problems)==0,problems=problems)
}
validate_stores(stores)
validate_products(products)
```

2.3 Alternative representations

Attribute	Option A	Option B	Final decision
Region	Character	Factor	Factor for a controlled list; character for free-form labels.
Promotion	'Yes'/'No' text	Logical TRUE/FALSE	Logical for reliable conditions.
Store ID	Integer	Character	Character to preserve prefixes and leading zeros.
Price	Integer cents	Double	Double in this teaching prototype; integer minor units are an option for strict financial systems.

2.4 Test cases

```
bad_stores <- stores
bad_stores$daily_footfall[2] <- -10
bad_stores$store_id[3] <- NA
validate_stores(bad_stores)

bad_products <- products
bad_products$unit_price[1] <- -5
validate_products(bad_products)

stopifnot(validate_stores(stores)$valid)
stopifnot(validate_products(products)$valid)
```

TASK 3 – CREATE A COERCION-SAFE SALES DATA PREPARATION MODULE

3.1 Automatic coercion

An atomic vector stores one underlying type. When mixed types are combined, R promotes the values to a common type. For example, a number combined with a character string generally produces a character vector. A list can retain different types in separate elements.

```
mixed_numeric <- c(10,20L,30)       # common numeric type
mixed_text <- c(10,"20",30)         # character vector
mixed_logical <- c(TRUE,1,FALSE)    # promoted to a common type
typeof(mixed_numeric)
typeof(mixed_text)
typeof(mixed_logical)

# Lists preserve heterogeneous values
safe_values <- list(units=10L, price=20.50,
                    product_id="P001", promoted=TRUE)
str(safe_values)
```

3.2 Safe preparation module

```
prepare_sales <- function(raw) {
  required <- c("store_id","product_id","units_sold",
                "unit_price","channel","month")
  missing_cols <- setdiff(required,names(raw))
  if(length(missing_cols))
    stop("Missing columns: ",paste(missing_cols,collapse=", "))

  x <- raw
  x$store_id <- as.character(x$store_id)
  x$product_id <- as.character(x$product_id)
  x$units_sold <- suppressWarnings(as.numeric(as.character(x$units_sold)))
  x$unit_price <- suppressWarnings(as.numeric(as.character(x$unit_price)))
  x$channel <- as.character(x$channel)
  x$month <- as.character(x$month)

  allowed <- c("in_store","online","mobile_app")
  x$missing_flag <- is.na(x$store_id) | is.na(x$product_id) |
                    is.na(x$units_sold) | is.na(x$unit_price)
  x$negative_flag <- (!is.na(x$units_sold) & x$units_sold<0) |
                     (!is.na(x$unit_price) & x$unit_price<0)
  x$channel_flag <- is.na(x$channel) | !(x$channel %in% allowed)
  x$invalid_flag <- x$missing_flag | x$negative_flag | x$channel_flag
  x$valid <- !x$invalid_flag
  x$revenue <- ifelse(x$valid,x$units_sold*x$unit_price,NA_real_)
  x
}
```

```
raw_test <- data.frame(
 store_id=c("S001","S002","S003","S004",NA),
 product_id=c("P001","P002","P001","P003","P004"),
 units_sold=c("5","7","-2","four","3"),
 unit_price=c("320","145.5","320","92","78"),
 channel=c("in_store","online","mobile_app","unknown","in_store"),
 month=c("Jan","Jan","Jan","Feb","Feb")
)
checked <- prepare_sales(raw_test)
checked[,c("store_id","units_sold","unit_price",
          "missing_flag","negative_flag","channel_flag","valid","revenue")]
```

3.3 Logical-vector operations

- & combines conditions that must both be true; | accepts either condition; ! reverses a condition.
- any() checks whether at least one element meets a condition; all() checks every element.
- which() returns the positions of TRUE values and identifies rejected rows.
- ifelse() assigns values according to a condition; check its output type when branches have different types.

```
positive_units <- checked$units_sold > 0
allowed_channel <- checked$channel %in%
                  c("in_store","online","mobile_app")
usable <- !checked$missing_flag &
         !checked$negative_flag & allowed_channel
any(!usable)
all(checked$valid)
which(!checked$valid)
checked$status <- ifelse(checked$valid,"Accepted","Rejected")
```

Test	Expected behaviour
5 units × 320; allowed channel	Accepted; revenue = 1600
Units = -2	Rejected by negative-value rule
Units = 'four'	Conversion yields NA; rejected
Price = -5	Rejected
Channel = unknown	Rejected by channel rule
Missing store ID	Rejected

TASK 4 – CONSTRUCT A STORE-BY-PRODUCT SALES MATRIX PROCESSING ENGINE

4.1 Matrix construction and operations

A matrix is a two-dimensional atomic structure. It is suitable for numeric measurements where rows and columns have stable meanings. Row names identify stores and column names identify products.

```
units_matrix <- matrix(
 c(10,8,5,2, 6,12,4,3, 9,7,11,5, 3,6,8,4, 4,9,2,1),
 nrow=5,ncol=4,byrow=TRUE,
 dimnames=list(stores$store_id,products$product_id)
)
units_matrix
units_matrix["S001","P001"]  # one cell
units_matrix[1,]            # one store
units_matrix[,"P002"]       # one product
rowSums(units_matrix)       # total units per store
colSums(units_matrix)       # total units per product
rowMeans(units_matrix)
t(units_matrix)             # transpose

price_vector <- products$unit_price
names(price_vector) <- products$product_id
price_vector <- price_vector[colnames(units_matrix)]
stopifnot(identical(names(price_vector),colnames(units_matrix)))
revenue_by_store <- drop(units_matrix %*% price_vector)
revenue_by_store
```

4.2 Matrix-to-data-frame conversion

```
matrix_long <- as.data.frame(as.table(units_matrix),
                             stringsAsFactors=FALSE)
names(matrix_long) <- c("store_id","product_id","units_sold")
matrix_long$units_sold <- as.integer(matrix_long$units_sold)
matrix_long <- merge(matrix_long,
                     products[,c("product_id","unit_price")],
                     by="product_id",all.x=TRUE)
matrix_long$revenue <- matrix_long$units_sold*matrix_long$unit_price
head(matrix_long)

stopifnot(nrow(units_matrix)==nrow(stores))
stopifnot(ncol(units_matrix)==nrow(products))
stopifnot(all(units_matrix>=0))
stopifnot(sum(units_matrix)==sum(matrix_long$units_sold))
```

The matrix is useful for repeated numerical calculations. The data frame is better when each observation needs metadata, joins, filtering or export. Conversion should occur at reporting boundaries, while the original IDs remain attached.

TASK 5 – CREATE A MULTI-DIMENSIONAL SALES ARRAY MODEL

5.1 Four-dimensional sales array

The array represents stores × products × months × channels. Each cell records units sold for a particular combination. Dimension names make array positions understandable.

```
set.seed(2026)
store_ids <- stores$store_id
product_ids <- products$product_id
months <- c("Jan","Feb","Mar")
channels <- c("in_store","online","mobile_app")

sales_array <- array(
 sample(0:20,5*4*3*3,replace=TRUE),
 dim=c(5,4,3,3),
 dimnames=list(store=store_ids,product=product_ids,
               month=months,channel=channels)
)
dim(sales_array)
sales_array["S001","P001","Jan","in_store"]
sales_array[,,"Jan","in_store"]

units_by_store <- apply(sales_array,1,sum)
units_by_product <- apply(sales_array,2,sum)
units_by_month <- apply(sales_array,3,sum)
units_by_channel <- apply(sales_array,4,sum)
average_by_channel <- apply(sales_array,4,mean)

channel_first <- aperm(sales_array,c(4,3,1,2))
array_long <- as.data.frame(as.table(sales_array))
names(array_long) <- c("store_id","product_id","month",
                       "channel","units_sold")
head(array_long)
```

An expression such as sales_array['S001','P001','Jan','in_store'] is easier to interpret than numeric-only indexing. A dense array reserves space for every combination, including zero-sales combinations. A long data frame or database table may be more efficient when sales are sparse.

TASK 6 – BUILD AN INTEGRATED LIST, FACTOR AND DATA-FRAME LAYER

6.1 Heterogeneous nested list

```
store_record <- list(
 identity=list(store_id="S001",store_name="Central"),
 location=list(region="North",city="Chennai"),
 operations=list(daily_footfall=420L,is_open=TRUE,
                 opening_hours="09:00-21:00"),
 performance=list(monthly_target=250000.00,
                  promotion_active=FALSE)
)
str(store_record)
store_record$identity$store_id
store_record$operations$daily_footfall
```

6.2 Factors and category rules

Variable	Representation	Reason
store_id / product_id	Character	Identifier, not a value for arithmetic.
Names	Character	Descriptive and potentially free-form.
region	Factor	Controlled list of regions.
store_format	Factor	Known store formats.
channel	Factor	Known sales channels.
stock_risk	Ordered factor	Low < Medium < High has a meaningful order.
promotion_active	Logical	Binary condition.

```
stock_risk <- factor(c("Low","Medium","High","Low","High"),
 levels=c("Low","Medium","High"),ordered=TRUE)
stock_risk[1] < stock_risk[3]
stores$region <- factor(stores$region,
 levels=c("North","South","East","West"))
products$category <- factor(products$category)
```

6.3 Integrated data-frame operations

```
store_summary <- data.frame(
 store_id=rownames(units_matrix),
 total_units=as.integer(rowSums(units_matrix)),
 revenue=as.numeric(revenue_by_store)
)
store_analysis <- merge(stores,store_summary,
                        by="store_id",all.x=TRUE)
open_stores <- subset(store_analysis,is_open==TRUE)

# Update a value by its stable key
```

```
stores$daily_footfall[stores$store_id=="S001"] <- 450L

# Add a new store while preserving factor levels
new_store <- data.frame(
 store_id="S006",store_name="South Point",
 region=factor("South",levels=levels(stores$region)),
 store_format=factor("High Street",levels=levels(stores$store_format)),
 daily_footfall=300L,is_open=TRUE
)
stores_updated <- rbind(stores,new_store)
aggregate(daily_footfall~region,data=stores,FUN=mean)

# Convert between representations
sales_wide <- xtabs(units_sold~store_id+product_id,data=matrix_long)
stopifnot(sum(units_matrix)==sum(matrix_long$units_sold))
```

Lists are appropriate for a single complex record. Data frames are appropriate for collections of observations. Factors are appropriate for controlled categories, but levels must be updated deliberately when a new valid category is introduced.

TASK 7 – DESIGN AN OBJECT-ORIENTED STORE/PRODUCT MODEL USING R CLASSES

7.1 S3 Store class

S3 is a lightweight R object-oriented mechanism. An S3 object can be a list with a class attribute. It is selected for the main prototype because it is simple and works naturally with base R.

```
new_store <- function(store_id,store_name,region,stock,reorder_level) {
 stopifnot(length(store_id)==1,!is.na(store_id))
 stopifnot(is.numeric(stock),length(stock)==1,stock>=0)
 stopifnot(is.numeric(reorder_level),length(reorder_level)==1,
         reorder_level>=0)
 structure(list(store_id=as.character(store_id),
              store_name=as.character(store_name),
              region=as.character(region),
              stock=as.integer(stock),
              reorder_level=as.integer(reorder_level)),
         class="RetailStore")
}
print.RetailStore <- function(x,...) {
 cat("Retail Store:",x$store_name,"\\n")
 cat("ID:",x$store_id," Region:",x$region,"\\n")
 cat("Stock:",x$stock," Reorder level:",x$reorder_level,"\\n")
 invisible(x)
}
update_stock <- function(x,quantity) {
```

```
 stopifnot(inherits(x,"RetailStore"))
 if(!is.numeric(quantity)||length(quantity)!=1) stop("Invalid quantity")
 next_stock <- x$stock+quantity
 if(next_stock<0) stop("Stock cannot become negative")
 x$stock <- as.integer(next_stock)
 x
}
needs_reorder <- function(x) {
 stopifnot(inherits(x,"RetailStore"))
 x$stock<=x$reorder_level
}
store_obj <- new_store("S001","Central","North",18,30)
print(store_obj)
store_obj <- update_stock(store_obj,20)
needs_reorder(store_obj)
```

7.2 Product class and method

```
new_product <- function(product_id,product_name,unit_price) {
 stopifnot(length(unit_price)==1,is.numeric(unit_price),
           !is.na(unit_price),unit_price>=0)
 structure(list(product_id=as.character(product_id),
                product_name=as.character(product_name),
                unit_price=as.double(unit_price)),
           class="RetailProduct")
}
calculate_revenue <- function(product,units) {
 stopifnot(inherits(product,"RetailProduct"))
 stopifnot(is.numeric(units),all(units>=0,na.rm=TRUE))
 units*product$unit_price
}
product_obj <- new_product("P001","Rice 5kg",320)
calculate_revenue(product_obj,c(1,3,5))
```

7.3 S4 comparison prototype

```
if(!methods::isClass("RetailProductS4")) {
 methods::setClass("RetailProductS4",
  slots=c(product_id="character",
          product_name="character",
          unit_price="numeric"),
  validity=function(object) {
   if(length(object@product_id)!=1 ||
      is.na(object@product_id) || object@product_id=="")
     return("Invalid product ID")
   if(length(object@unit_price)!=1 || object@unit_price<0)
     return("Price must be non-negative")
```

```
  TRUE
 })
}
s4_product <- methods::new("RetailProductS4",
 product_id="P010",product_name="Example item",unit_price=50)
methods::validObject(s4_product)
```

Feature	S3	S4	Reference Class
Structure	List + class attribute	Formal slots/class	Mutable reference object
Validation	Constructor/functions	Formal validity method	Custom methods/field checks
Complexity	Low	Moderate	Moderate
Mutation	Usually return modified copy	Formal object operations	Shared reference semantics
Best use	Flexible business prototype	Strict schema enforcement	Shared mutable state

The final student prototype uses S3 because it is lightweight and readable. S4 is preferable where formal slot validation is mandatory. Reference Classes are not selected because shared mutable state can make changes harder to trace.

TASK 8 – DEVELOP A REUSABLE R FUNCTION LIBRARY

8.1 Reusable functions

```
# 1. Validation function
validate_nonnegative <- function(x,field_name="value",allow_na=FALSE) {
 if(!is.numeric(x))
  return(list(valid=FALSE,message=paste(field_name,"must be numeric")))
 if(!allow_na && anyNA(x))
  return(list(valid=FALSE,message=paste(field_name,"contains NA")))
 if(any(x<0,na.rm=TRUE))
  return(list(valid=FALSE,message=paste(field_name,"cannot be negative")))
 list(valid=TRUE,message="Valid")
}

# 2. Revenue by store
calculate_store_revenue <- function(units,prices) {
 if(!is.matrix(units)||!is.numeric(units))
  stop("units must be a numeric matrix")
 if(is.null(colnames(units))) stop("Product column names are required")
 prices <- prices[colnames(units)]
 if(anyNA(prices)) stop("Missing price for a product")
 drop(units %*% prices)
}

# 3. Sales summary
```

```
summarise_sales <- function(data,group_by="store_id") {
 required <- c(group_by,"units_sold","revenue")
 if(!all(required %in% names(data))) stop("Required columns missing")
 aggregate(data[c("units_sold","revenue")],
          by=list(group=data[[group_by]]),FUN=sum,na.rm=TRUE)
}

# 4. Data quality report
quality_report <- function(data) {
 list(row_count=nrow(data),column_count=ncol(data),
      missing_by_column=vapply(data,function(z)sum(is.na(z)),integer(1)),
      duplicate_rows=sum(duplicated(data)),
      structure=capture.output(str(data)))
}

# 5. Inventory update
adjust_inventory <- function(current_stock,change=0L) {
 if(!is.numeric(current_stock)||!is.numeric(change))
  stop("Stock and change must be numeric")
 updated <- current_stock+change
 if(any(updated<0,na.rm=TRUE)) stop("Stock cannot become negative")
 as.integer(updated)
}
```

8.2 Function tests and contracts

```
validate_nonnegative(c(1,2,3),"footfall")
validate_nonnegative(c(1,-2),"price")
validate_nonnegative(c(1,NA),"stock")
calculate_store_revenue(units_matrix,price_vector)
summarise_sales(matrix_long,"store_id")
quality_report(matrix_long)
adjust_inventory(c(10L,20L),c(-2L,5L))
stopifnot(validate_nonnegative(0,"price")$valid)
stopifnot(!validate_nonnegative(-1,"price")$valid)
stopifnot(identical(adjust_inventory(5L,-5L),0L))
try(adjust_inventory(5L,-6L))
```

Function	Arguments	Return value	Expected behaviour
validate_nonnegative	Numeric vector, field, allow_na	List(valid, message)	Reject wrong type, disallowed NA and negatives.
calculate_store_revenue	Numeric matrix, named prices	Numeric vector	Align price by product ID and calculate store revenue.
summarise_sales	Data frame, grouping column	Data frame	Sum units and revenue by group.
quality_report	Data frame	List	Dimensions, missing counts, duplicate count and structure.

adjust_inventory	Current stock, change	Integer vector	Prevent negative inventory.

TASK 9 – INTEGRATE, TEST AND REFINE THE COMPLETE FRAMEWORK

9.1 Integrated workflow

7. Create store and product master tables.
8. Receive sales as raw data and do not assume imported types are correct.
9. Run coercion-safe preparation and separate accepted and rejected records.
10. Check store and product IDs against master tables.
11. Calculate revenue only for accepted records.
12. Aggregate units into a store-by-product matrix.
13. Populate the sales array for month and channel analysis.
14. Create quality and management summaries.

9.2 Integrated example

```
sales_feed <- data.frame(
 store_id=c("S001","S001","S002","S003","S005","S002"),
 product_id=c("P001","P002","P001","P003","P004","P004"),
 units_sold=c(5,3,8,4,6,2),
 unit_price=c(320,145.5,320,92,78,78),
 channel=c("in_store","online","in_store","mobile_app","online","in_store"),
 month=c("Jan","Jan","Jan","Feb","Feb","Mar")
)
prepared <- prepare_sales(sales_feed)
accepted_sales <- subset(prepared,valid)
rejected_sales <- subset(prepared,!valid)

accepted_sales$known_store <-
 accepted_sales$store_id %in% stores$store_id
accepted_sales$known_product <-
 accepted_sales$product_id %in% products$product_id
accepted_sales$reference_valid <-
 accepted_sales$known_store & accepted_sales$known_product
accepted_sales <- subset(accepted_sales,reference_valid)
accepted_sales$revenue <-
 accepted_sales$units_sold*accepted_sales$unit_price

sales_report <- aggregate(cbind(units_sold,revenue)~store_id,
                          data=accepted_sales,FUN=sum)
sales_report
```

9.3 Systematic test plan

ID	Input / condition	Expected result	Pass condition
T01	Required columns present	Preparation completes	No missing-column error
T02	Missing store ID	Row rejected	missing_flag TRUE
T03	Negative units	Row rejected	negative_flag TRUE
T04	Text in numeric field	Conversion yields NA and row rejected	valid FALSE
T05	Unknown channel	Row rejected	channel_flag TRUE
T06	Unknown store ID	Reference integrity fails	known_store FALSE
T07	Unknown product ID	Reference integrity fails	known_product FALSE
T08	Negative inventory update	Error raised	No negative stock
T09	Matrix dimension mismatch	Dimension check fails	stopifnot detects
T10	Array dimensions named	All dimensions labelled	dimnames not NULL

9.4 Failure and refinement log

Potential failure	Cause	Refinement
IDs become numeric	Import/coercion	Import IDs as character.
Text numeric becomes NA	Unparseable text	Check NA after explicit conversion.
Incorrect revenue	Price order differs from matrix columns	Align by product ID and assert names.
New factor category becomes NA	Unknown factor level	Validate categories and manage levels centrally.
Array indexing misunderstood	Dimension order unclear	Use named dimensions and documented order.
Negative stock	Uncontrolled decrement	Validate before committing update.

A correct integrated workflow must show both accepted and rejected rows and retain the reason for rejection. Invalid data should not silently enter the analytical layer.

TASK 10 – EVIDENCE-BASED SCALABILITY AND MAINTAINABILITY EVALUATION

10.1 Benchmark experiment

The benchmark measures matrix construction time, row/column summary time and object size for 500 × 40, 10,000 × 40 and 10,000 × 100. Results depend on the computer and R version; run the code and record the actual console output. Do not report invented timings.

```
benchmark_matrix <- function(n_stores,n_products,seed=2026) {
 set.seed(seed)
 store_names <- sprintf("S%05d",seq_len(n_stores))
 product_names <- sprintf("P%03d",seq_len(n_products))
 t_build <- system.time({
  x <- matrix(sample(0:20,n_stores*n_products,replace=TRUE),
              nrow=n_stores,ncol=n_products,
```

```
            dimnames=list(store_names,product_names))
 })
 t_summary <- system.time({
  by_store <- rowSums(x)
  by_product <- colSums(x)
 })
 list(stores=n_stores,products=n_products,cells=length(x),
      build_seconds=unname(t_build["elapsed"]),
      summary_seconds=unname(t_summary["elapsed"]),
      object_size_bytes=as.numeric(object.size(x)),
      checksum=sum(by_store))
}
results <- do.call(rbind,list(
 as.data.frame(benchmark_matrix(500,40)),
 as.data.frame(benchmark_matrix(10000,40)),
 as.data.frame(benchmark_matrix(10000,100))
))
print(results)

barplot(results$object_size_bytes/1024^2,
 names.arg=c("500 x 40","10,000 x 40","10,000 x 100"),
 ylab="Object size (MiB)",main="Matrix memory use by scenario")
barplot(results$summary_seconds,
 names.arg=c("500 x 40","10,000 x 40","10,000 x 100"),
 ylab="Elapsed seconds",main="Summary execution time")
```

10.2 Structural comparison

Scenario	Stores	Products	Cells	Raw double payload
A	500	40	20,000	0.153 MiB
B	10,000	40	400,000	3.052 MiB
C	10,000	100	1,000,000	7.629 MiB

The cell count grows linearly with stores and categories. Growth from 500 × 40 to 10,000 × 40 creates 20 times as many cells. Growth from 10,000 × 40 to 10,000 × 100 creates 2.5 times as many cells. object.size() will be greater than the raw payload estimate.

10.3 Refinements

- Use vectorised operations such as rowSums(), colSums() and matrix multiplication rather than nested R loops.
- Avoid repeated conversions among matrix, array and data frame inside tight loops.
- Keep the validated long-form sales table as the auditable source when transactions are sparse or need metadata.

- Generate matrix and array views for analytical tasks.
- For production, consider a database or columnar storage and use R for analysis.

TASK 11 – DEFEND THE FINAL ARCHITECTURE UNDER A CHANGED REQUIREMENT

11.1 Revised requirement

The chain now requires 10,000 stores, 100 product categories, three sales channels and frequent stock and price updates. The framework must remain easy to validate, transform and extend.

11.2 Architecture decisions

Component	Decision	Reason
Character IDs	Retain	Stable identifiers remain essential.
Master data frames	Retain and extend	Separate descriptive data and support joins.
Factors	Modify	Centralise approved category levels.
Store-product matrix	Retain as analytical view	Efficient 10,000 × 100 numeric summaries.
Sales array	Redesign to 4 dimensions	Add channel to store × product × month.
Raw ingestion	Strengthen	Explicit parsing, rejection log and reference checks.
S3 model	Retain	Reusable business operations.
Persistence	Extend	Database or partitioned files for history and frequent updates.

11.3 Revised array capacity

```
n_store <- 10000
n_product <- 100
n_month <- 12
n_channel <- 3
required_cells <- n_store*n_product*n_month*n_channel
required_bytes <- required_cells*8
required_MiB <- required_bytes/1024^2
c(cells=required_cells,bytes=required_bytes,MiB=required_MiB)

# Validate the four-dimensional design with a small example
small_demo <- array(0,dim=c(3,4,2,3),
 dimnames=list(store=c("S1","S2","S3"),
               product=c("P1","P2","P3","P4"),
               month=c("Jan","Feb"),
               channel=c("in_store","online","mobile_app")))
dim(small_demo)
```

11.4 Safe frequent updates

```
update_product_price <- function(product_table,id,new_price) {
 if(length(new_price)!=1 || !is.numeric(new_price) ||
```

```
   is.na(new_price) || new_price<0)
  stop("Price must be one non-negative numeric value")
 i <- match(id,product_table$product_id)
 if(is.na(i)) stop("Unknown product ID")
 product_table$unit_price[i] <- as.double(new_price)
 product_table
}

update_stock_table <- function(inventory,store_id,product_id,change) {
 i <- which(inventory$store_id==store_id &
           inventory$product_id==product_id)
 if(length(i)!=1) stop("Expected exactly one inventory record")
 next_stock <- inventory$stock_units[i]+change
 if(next_stock<0) stop("Stock cannot become negative")
 inventory$stock_units[i] <- as.integer(next_stock)
 inventory
}
```

11.5 Remaining trade-offs and future improvements

- Dense arrays are easy to calculate with but can use considerable memory when most combinations have no sales; sparse tables or database aggregation may be better.
- Factors provide consistent categories but require deliberate level management when new valid categories are introduced.
- Double precision is convenient; strict financial applications may prefer integer minor currency units or decimal arithmetic.
- S3 is flexible but less formal than S4; S4 can be used at critical validated interfaces.
- The prototype is in-memory; production should add persistent storage, access control, audit logging, transaction handling and automated tests.

TASK 12 – DOCUMENT, PACKAGE AND PRESENT THE FINAL FRAMEWORK

12.1 Data dictionary

Object / field	Class / typeof	Dimensions / length	Purpose	Validation
stores$store_id	character / character	n stores	Unique store key	Not missing, nonblank, unique
stores$store_name	character / character	n stores	Display name	Not missing
stores$region	factor / integer codes	n stores	Controlled region	Approved levels
stores$daily_footfall	integer / integer	n stores	Daily visitors	Numeric, >= 0
stores$is_open	logical / logical	n stores	Open status	TRUE/FALSE; review NA
products$product_id	character / character	n products	Product key	Not missing, unique
products$unit_price	numeric / double	n products	Selling price	Numeric, >= 0
products$reorder_level	integer / integer	n products	Reorder threshold	Integer, >= 0
units_matrix	matrix / numeric	stores × products	Units summary	Non-negative; named dimensions
sales_array	array / numeric	store × product × month × channel	Sales cube	Named dimensions and non-negative values
store_record	list / list	Nested elements	Heterogeneous record	Required elements present
matrix_long	data.frame / list columns	records × fields	Long-form sales table	IDs map to master tables
store_obj	RetailStore / list	One object	Store business entity	Constructor and update checks

12.2 Object inspection

```
class(stores); typeof(stores$store_id); dim(stores); length(stores$store_id); str(stores)
class(units_matrix); typeof(units_matrix); dim(units_matrix); length(units_matrix)
class(sales_array); typeof(sales_array); dim(sales_array); length(sales_array)
dimnames(sales_array)
sessionInfo()
```

12.3 Summary-report function

```
create_framework_report <- function(stores,products,sales_data,matrix_data) {
 list(
  generated_at=Sys.time(),
  store_count=nrow(stores),
  open_store_count=sum(stores$is_open,na.rm=TRUE),
  product_count=nrow(products),
  category_count=length(unique(products$category)),
  sales_record_count=nrow(sales_data),
  valid_sales_count=sum(sales_data$valid,na.rm=TRUE),
  rejected_sales_count=sum(!sales_data$valid,na.rm=TRUE),
  total_units_in_matrix=sum(matrix_data,na.rm=TRUE),
  total_revenue=sum(sales_data$revenue,na.rm=TRUE),
  store_validation=validate_stores(stores),
  product_validation=validate_products(products),
```

```
  session=R.version.string
 )
}
final_report <- create_framework_report(stores,products,prepared,units_matrix)
str(final_report)
```

12.4 Suggested script package

File	Contents
00_setup.R	Clear workspace, set seed, record R version and packages.
01_master_data.R	Store/product tables and validation.
02_sales_validation.R	Coercion-safe import and record checks.
03_matrix_array.R	Matrix and array construction and calculations.
04_classes_functions.R	S3/S4 prototypes and reusable functions.
05_integration_tests.R	End-to-end workflow and test cases.
06_benchmark_report.R	Scalability benchmark, graphs and summary report.

12.5 Management recommendation

Management should use character vectors for store/product IDs, logical vectors for binary operational flags, numeric vectors for quantities and prices, and factors for controlled categories. Store and product master information should be maintained in data frames. Nested lists are useful for one complex store record. Matrices should be generated as analytical views for store-by-product calculations. Named arrays should support multi-dimensional summaries across stores, products, months and channels. S3 classes can package business operations such as stock updates and revenue calculation. The validated long-form sales table should remain the auditable source of sales facts.

12.6 Lessons learned

15. Automatic coercion is convenient but can damage meaning; explicit conversion and validation are essential.

16. Matrices and arrays are efficient for numeric calculations but do not replace data frames for metadata and joins.

17. Dimension names and stable IDs prevent errors when data are reordered or expanded.

18. Scalability must be evaluated using reproducible experiments; actual sizes and timings depend on the runtime environment.

CONSOLIDATED REPRODUCIBLE R SCRIPT

The following compact script provides the central end-to-end execution order. The individual task sections above contain expanded code and explanation. Save this script as smart_retail_assignment1.R and run it from a fresh R session.

```
# SMART RETAIL R DATA FRAMEWORK - BASE R
rm(list=ls())
```

```
set.seed(2026)
options(stringsAsFactors=FALSE)

stores <- data.frame(
 store_id=c("S001","S002","S003","S004","S005"),
 store_name=c("Central","Marina","Airport","City Mall","West End"),
 region=factor(c("North","South","East","South","West"),
               levels=c("North","South","East","West")),
 daily_footfall=as.integer(c(420,510,680,390,275)),
 is_open=c(TRUE,TRUE,TRUE,FALSE,TRUE)
)
products <- data.frame(
 product_id=c("P001","P002","P003","P004"),
 product_name=c("Rice 5kg","Oil 1L","Tea 250g","Soap Pack"),
 category=factor(c("Grocery","Grocery","Beverage","Personal Care")),
 unit_price=as.double(c(320,145.5,92,78)),
 reorder_level=as.integer(c(30,25,20,40))
)

validate_stores <- function(x) {
 p <- character()
 if(anyNA(x$store_id)||anyDuplicated(x$store_id)) p<-c(p,"Invalid IDs")
 if(any(x$daily_footfall<0,na.rm=TRUE)) p<-c(p,"Negative footfall")
 if(!is.logical(x$is_open)) p<-c(p,"Open flag must be logical")
 list(valid=length(p)==0,problems=p)
}
validate_products <- function(x) {
 p <- character()
 if(anyNA(x$product_id)||anyDuplicated(x$product_id)) p<-c(p,"Invalid IDs")
 if(any(x$unit_price<0,na.rm=TRUE)) p<-c(p,"Negative price")
 list(valid=length(p)==0,problems=p)
}
stopifnot(validate_stores(stores)$valid)
stopifnot(validate_products(products)$valid)

prepare_sales <- function(raw) {
 x<-raw
 x$store_id<-as.character(x$store_id)
 x$product_id<-as.character(x$product_id)
 x$units_sold<-suppressWarnings(as.numeric(as.character(x$units_sold)))
 x$unit_price<-suppressWarnings(as.numeric(as.character(x$unit_price)))
 x$channel<-as.character(x$channel)
 x$missing_flag<-is.na(x$store_id)|is.na(x$product_id)|
                 is.na(x$units_sold)|is.na(x$unit_price)
 x$negative_flag<-(!is.na(x$units_sold)&x$units_sold<0)|
                  (!is.na(x$unit_price)&x$unit_price<0)
```

```
 x$channel_flag<-is.na(x$channel)|
              !(x$channel%in%c("in_store","online","mobile_app"))
 x$valid<-!(x$missing_flag|x$negative_flag|x$channel_flag)
 x$revenue<-ifelse(x$valid,x$units_sold*x$unit_price,NA_real_)
 x
}
sales_feed<-data.frame(
 store_id=c("S001","S001","S002","S003","S005","S002"),
 product_id=c("P001","P002","P001","P003","P004","P004"),
 units_sold=c("5","3","8","4","6","2"),
 unit_price=c("320","145.5","320","92","78","78"),
 channel=c("in_store","online","in_store","mobile_app","online","in_store"),
 month=c("Jan","Jan","Jan","Feb","Feb","Mar")
)
prepared<-prepare_sales(sales_feed)
accepted<-subset(prepared,valid)
accepted<-subset(accepted,
 store_id%in%stores$store_id & product_id%in%products$product_id)

units_matrix<-matrix(
 c(10,8,5,2,6,12,4,3,9,7,11,5,3,6,8,4,4,9,2,1),
 nrow=5,byrow=TRUE,
 dimnames=list(stores$store_id,products$product_id)
)
price_vector<-products$unit_price
names(price_vector)<-products$product_id
price_vector<-price_vector[colnames(units_matrix)]
revenue_by_store<-drop(units_matrix%*%price_vector)

months<-c("Jan","Feb","Mar")
channels<-c("in_store","online","mobile_app")
sales_array<-array(sample(0:20,5*4*3*3,replace=TRUE),
 dim=c(5,4,3,3),
 dimnames=list(store=stores$store_id,product=products$product_id,
               month=months,channel=channels))
array_month_totals<-apply(sales_array,3,sum)
array_channel_totals<-apply(sales_array,4,sum)

matrix_long<-as.data.frame(as.table(units_matrix))
names(matrix_long)<-c("store_id","product_id","units_sold")
matrix_long$units_sold<-as.integer(matrix_long$units_sold)
matrix_long<-merge(matrix_long,products[,c("product_id","unit_price")],
                   by="product_id")
matrix_long$revenue<-matrix_long$units_sold*matrix_long$unit_price

new_store<-function(id,name,region,stock,reorder) {
```

```
 if(stock<0||reorder<0) stop("Stock and reorder must be non-negative")
 structure(list(store_id=id,store_name=name,region=region,
               stock=as.integer(stock),reorder_level=as.integer(reorder)),
          class="RetailStore")
}
update_stock<-function(x,change) {
 if(x$stock+change<0) stop("Negative stock not allowed")
 x$stock<-as.integer(x$stock+change); x
}
sample_store<-new_store("S001","Central","North",18,30)
sample_store<-update_stock(sample_store,5)

quality_report<-function(x) {
 list(rows=nrow(x),columns=ncol(x),
      missing=vapply(x,function(z)sum(is.na(z)),integer(1)),
      duplicates=sum(duplicated(x)))
}
framework_report<-list(
 stores=nrow(stores),products=nrow(products),
 accepted_sales=nrow(accepted),
 rejected_sales=sum(!prepared$valid),
 revenue_by_store=revenue_by_store,
 month_totals=array_month_totals,
 channel_totals=array_channel_totals,
 quality=quality_report(prepared),
 session=sessionInfo()
)
print(framework_report)
cat("Matrix bytes:",object.size(units_matrix),"\\n")
cat("Array bytes:",object.size(sales_array),"\\n")

benchmark_matrix<-function(ns,np) {
 set.seed(2026)
 tb<-system.time(x<-matrix(sample(0:20,ns*np,TRUE),nrow=ns))
 ts<-system.time(r<-rowSums(x);cc<-colSums(x))
 data.frame(stores=ns,products=np,cells=length(x),
  build=unname(tb["elapsed"]),summary=unname(ts["elapsed"]),
  bytes=as.numeric(object.size(x)))
}
benchmark_results<-rbind(benchmark_matrix(500,40),
                         benchmark_matrix(10000,40),
                         benchmark_matrix(10000,100))
print(benchmark_results)
sessionInfo()
# END
```

CONCLUSION

The framework assigns each type of retail information to a suitable R representation and demonstrates how the structures cooperate. Vectors hold consistent attributes; lists hold heterogeneous records; factors encode controlled categories; data frames provide relational and analysis-ready tables; matrices support two-dimensional numeric operations; arrays support multi-dimensional sales analysis; and S3/S4 mechanisms provide structured business objects.

The key design principle is to preserve meaning. IDs should not become numbers, missing values should not be confused with zero, categories should be validated before factor conversion, and matrix/array positions should retain meaningful names. The validated long-form sales table acts as the auditable source, while matrices and arrays are generated for analysis.

REFERENCES

19. R Core Team. R: A Language and Environment for Statistical Computing. https://www.r-project.org/

20. R Documentation. c – Combine Values into a Vector or List. https://stat.ethz.ch/R-manual/R-devel/library/base/html/c.html

21. R Documentation. data.frame – Data Frames. https://stat.ethz.ch/R-manual/R-devel/library/base/html/data.frame.html

22. R Documentation. matrix – Matrices. https://stat.ethz.ch/R-manual/R-devel/library/base/html/matrix.html

23. R Documentation. array – Multi-way Arrays. https://stat.ethz.ch/R-manual/R-devel/library/base/html/array.html

24. R Documentation. factor – Factors. https://stat.ethz.ch/R-manual/R-devel/library/base/html/factor.html

25. R Documentation. class – Object Classes. https://stat.ethz.ch/R-manual/R-devel/library/base/html/class.html

26. R Documentation. setClass – S4 Class Definition. https://stat.ethz.ch/R-manual/R-devel/library/methods/html/setClass.html

27. R Documentation. object.size – Report the Space Allocated for an Object. https://stat.ethz.ch/R-manual/R-devel/library/utils/html/object.size.html

28. R Documentation. system.time – CPU Time Used. https://stat.ethz.ch/R-manual/R-devel/library/base/html/system.time.html